Architettura degli Elaboratori a.a. 2024/25

Appello d'esame 18/02/2026 (ASM)

Operazioni preliminari

1. Aprire il file `AE - Uso del computer in laboratorio per la prova ASM.pdf`, contenente informazioni utili per lo svolgimento di questa prova.
2. Aprire tramite RARS il file `program01.asm` in questa directory
3. Completare le seguenti righe con i propri dati in `program01.asm`

```
########################################
# INSERIRE I PROPRI DATI QUI:
# Nome:
# Cognome:
# Matricola:
########################################
```

Esercizio

Si realizzi in assembly RISC-V il seguente programma. Sia data in input da utente una stringa di caratteri `S`. La stringa è lunga al più 24 caratteri (non è necessario verificare questo vincolo) e consiste di numeri esadecimali rappresentati secondo il codice ASCII: '`0`' (codice ASCII `0x30`), '`1`' (codice ASCII `0x31`), ..., '`9`' (codice ASCII `0x39`), '`A`' (codice ASCII `0x41`), '`B`' (codice ASCII `0x42`), ..., '`F`' (codice ASCII `0x46`). La sequenza termina quando viene letto il carattere `'\n'` (codice ASCII `0xA`). Il programma deve generare un numero considerando solo i bit corrispondenti alle singole cifre della stringa `S`. Il programma deve:

1. stampare a schermo la rappresentazione esadecimale del numero immesso tramite la stringa `S` e
2. stampare a schermo su una nuova riga la rappresentazione esadecimale del numero generato.

Se la sequenza `S` è completamente vuota, non dovrà essere scritto a schermo alcunché. Ogni numero è formato al più da otto caratteri. Se, ad esempio, `S` è `120A4` allora dovrà essere scritto a schermo su due linee separate:

```
0x000120a4
0x00000004
```

perché la rappresentazione binaria del numero `120A4` è la seguente `0001 0010 0000 1010 0100` e selezionando solo i bit 1, 2, 0, a, e 4 si ottiene il numero `0x4`, perché il bit in posizione 2 di `120A4` è 1 e tutti gli altri sono zero.

Nella sezione dedicata al **text segment**, il programma deve avere nel comparto *main* il caricamento dei dati da input (già fornito nel file `program01.asm`: l'indirizzo base della stringa `S` verrà caricato nel registro `a0`), e il codice necessario a calcolare i valori e visualizzare i risultati sullo schermo. Si suggerisce di creare una maschera con i bit da selezionare nel numero.

Note: Commentare le righe più importanti del codice avendo cura di spiegare a cosa servano i registri. Si ricorda che la syscall per la stampa di un intero in rappresentazione esadecimale prevede come argomenti `a7=34` e `a0` deve contenere il numero che si vuole stampare a schermo, mentre la syscall che per la stampa di un carattere prevede come argomenti `a7=11` e `a0` deve contenere la

rappresentazione ascii del carattere che si vuole stampare a schermo . Si ricorda che la syscall per la stampa a intero può essere chiamata più volte per stampare due interi uno dopo l'altro.

Risultato atteso

Per ogni input al programma va stampato l'output della procedura suddetta seguito da accapo come spiegato precedentemente.

Ad esempio, il file `test-03.in` contiene il seguente input: '`FA01D3`', il suo output atteso (all'interno di `test-02.expt`) è:

```
0x00fa01d3
0x00000003
```

È possibile studiare i casi di test aprendo i file di input (nel formato `test-`*xy*`.in`, dove *xy* è un numero a due cifre) e output atteso (estensione `test-`*xy*`.expt`), confrontandoli col proprio (estensione `test-`*xy*`.out`).

Verifica di corretta esecuzione dell'esercizio

Per verificare che l'esercizio sia stato completato correttamente eseguire `run.sh` (doppio click sul file dal file manager, oppure esecuzione del comando `./run.sh` da terminale) e visualizzare il risultato aprendo il file `test_results.html`. Per ulteriori informazioni, consultare il file `AE - Uso del computer in laboratorio per la prova ASM.pdf`.

Bonus

Una soluzione pienamente funzionante, dove ogni test verrà completato usando meno di 127 istruzioni sarà premiata con un bonus di **1 punto** sul voto finale.

Note

- Non è consentito modificare il *data segment*.
- Il limite massimo di istruzioni eseguibili è `305900`. Oltre quel numero, l'esecuzione viene automaticamente terminata.
- Il file `program01.vuoto.asm` contiene una copia del file `program01.asm` che può essere utile in caso sia necessario ripartire da capo.
- Attenzione a non eseguire loop infiniti con leak della memoria: RARS potrebbe crashare e cancellare il file che state scrivendo, sentitevi liberi di salvare un file di backup prima di eseguire del codice rischioso.